

AI ASSISTED CODING

END LAB EXAM

NAME:P.NISHANTH

HTNO:2403A52066

BATCH:03

Q1:

TASK 1 :

Use AI to suggest fusion models and feature engineering steps

PROMPT:

Write Python code that loads or creates dummy temperature, CO₂, and video-count sensor data, normalizes them, adds simple feature-engineering (like rate of change or moving average), aligns timestamps, and shows an example of early fusion for a machine-learning model. Add brief comments explaining the fusion idea."

CODE:

```

q1task1.py > ...
1  import pandas as pd
2  import numpy as np
3  from sklearn.preprocessing import StandardScaler
4  from sklearn.ensemble import RandomForestRegressor
5  # 1. Create Dummy Sensor Data
6  def create_dummy_data():
7      ts = pd.date_range("2024-01-01 10:00", periods=10, freq="T")
8
9      temp = pd.DataFrame({
10         "timestamp": ts,
11         "temperature": np.random.uniform(20, 30, 10)
12     })
13
14     co2 = pd.DataFrame({
15         "timestamp": ts,
16         "co2": np.random.uniform(350, 600, 10)
17     })
18
19     video = pd.DataFrame({
20         "timestamp": ts,
21         "video_count": np.random.randint(5, 25, 10)
22     })
23     return temp, co2, video
24 # 2. Align Sensors by Timestamp
25 def align_sensors(temp, co2, video):
26     df = temp.merge(co2, on="timestamp").merge(video, on="timestamp")
27     df = df.set_index("timestamp").ffill().bfill()
28     return df
29 # 3. Feature Engineering
30 def feature_engineering(df):

```

```

q1task1.py > ...
30 def feature_engineering(df):
31     df["temp_change"] = df["temperature"].diff().fillna(0)
32     df["co2_change"] = df["co2"].diff().fillna(0)
33     df["count_ma3"] = df["video_count"].rolling(3).mean().fillna(df["video_count"].iloc[0])
34     return df
35 # 4. Normalize Data
36 def normalize(df):
37     scaler = StandardScaler()
38     norm = pd.DataFrame(scaler.fit_transform(df), columns=df.columns, index=df.index)
39     return norm
40 # 5. Early Fusion Example
41 def early_fusion_example(df):
42     # Using temperature to predict CO2 (example only)
43     X = df.drop("co2", axis=1)
44     y = df["co2"]
45     model = RandomForestRegressor()
46     model.fit(X, y)
47     print("\nModel trained using early fusion features.\n")
48     return model
49 # Main
50 if __name__ == "__main__":
51     temp, co2, video = create_dummy_data()
52     df = align_sensors(temp, co2, video)
53     df = feature_engineering(df)
54     df = normalize(df)
55     print("=== FUSED FEATURE DATA ===")
56     print(df.head())
57     early_fusion_example(df)
58

```

OUTPUT:

```
=== FUSED FEATURE DATA ===
timestamp      temperature      co2  video_count  temp_change  co2_change  count_ma3
2024-01-01 10:00:00    0.675528 -0.126755    0.606627    0.001986   -0.095943    1.003322
2024-01-01 10:01:00   -0.152244 -0.596622   -0.292080   -0.618148   -0.407783    1.003322
2024-01-01 10:02:00    0.437914 -0.133769   -1.190786    0.444110    0.211242   -1.185744
2024-01-01 10:03:00    0.681492  0.011236    0.381950    0.184465    0.000293   -1.368167
2024-01-01 10:04:00    1.119184 -1.263606    0.831303    0.329888   -0.942027   -0.456056

Model trained using early fusion features.

PS D:\AICODE> 
```

OBSERVATION:

After running the code, we obtain a fused dataset that combines temperature, CO₂, and video-based people-count data into a single aligned time-series. All sensor streams are normalized to a common scale, which makes them comparable and suitable for machine-learning models

TASK:2

Implement preprocessing pipeline and unit tests

PROMPT:

"Generate Python code for a preprocessing pipeline that loads temperature, CO₂, and video-count sensor data, handles missing values, normalizes each sensor stream, and outputs a fused dataset. Also generate unit tests that check normalization ranges and data consistency."

CODE:

```

import pandas as pd
from sklearn.preprocessing import StandardScaler, MinMaxScaler

class MultiSensorPreprocessor:
    def __init__(self): # FIXED
        self.temp_scaler = StandardScaler()
        self.co2_scaler = StandardScaler()
        self.count_scaler = MinMaxScaler()
    def fit(self, df):
        self.temp_scaler.fit(df[['temperature']])
        self.co2_scaler.fit(df[['co2']])
        self.count_scaler.fit(df[['video_count']])
    def transform(self, df):
        df = df.copy()
        df['temperature_norm'] = self.temp_scaler.transform(df[['temperature']])
        df['co2_norm'] = self.co2_scaler.transform(df[['co2']])
        df['video_norm'] = self.count_scaler.transform(df[['video_count']])
        return df
    def fit_transform(self, df):
        self.fit(df)
        return self.transform(df)

# Unit Tests
import unittest
class TestPreprocessing(unittest.TestCase):
    def setUp(self):
        self.df = pd.DataFrame({
            'temperature': [22, 23, 24],
            'co2': [400, 420, 450],
            'video_count': [5, 10, 20]
        })
        self.pre = MultiSensorPreprocessor()
        self.result = self.pre.fit_transform(self.df)
    def test_columns_exist(self):
        self.assertIn('temperature_norm', self.result.columns)
        self.assertIn('co2_norm', self.result.columns)

```

```

30         self.pre = MultiSensorPreprocessor()
31         self.result = self.pre.fit_transform(self.df)
32     def test_columns_exist(self):
33         self.assertIn('temperature_norm', self.result.columns)
34         self.assertIn('co2_norm', self.result.columns)
35         self.assertIn('video_norm', self.result.columns)
36     def test_video_range(self):
37         self.assertTrue(self.result['video_norm'].between(0, 1).all())
38 if __name__ == "__main__": # FIXED
39     df = pd.DataFrame({
40         'temperature': [22, 23, 24],
41         'co2': [400, 420, 450],
42         'video_count': [5, 10, 20]
43     })
44     pre = MultiSensorPreprocessor()
45     result = pre.fit_transform(df)
46     # PRINT OUTPUT TABLE
47     print(result.to_string(index=False))
48     # Run tests
49     unittest.main(exit=False)
50

```

OUTPUT:

temperature	co2	video_count	temperature_norm	co2_norm	video_norm
22	400	5	-1.224745	-1.135550	0.000000
23	420	10	0.000000	-0.162221	0.333333
24	450	20	1.224745	1.297771	1.000000

OBSERVATION:

The preprocessing pipeline successfully normalized all three sensor streams and generated new columns: `temperature_norm`, `co2_norm`, and `video_norm`. The unit tests confirmed that the normalized columns are created and that the video count normalization stays within the 0–1 range. This verifies the correctness of the preprocessing logic.

Q2:

TASK1:

Use AI to propose statistical and ML hybrid detectors

PROMPT:

Propose statistical + ML hybrid anomaly detectors for fused sensor features and generate Python code that demonstrates a hybrid anomaly detection approach using statistical filtering + ML model.

CODE:

```
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

# Sample fused features
X = np.array([
    [22, 400, 5],
    [23, 420, 10],
    [24, 430, 11],
    [25, 440, 12],
    [40, 600, 50] # anomaly
], dtype=float)

# 1. Statistical filter (z-score)
zs = (X - X.mean(axis=0)) / X.std(axis=0)
stat_flags = (np.abs(zs) > 2.5).any(axis=1).astype(float)

# 2. ML model (Isolation Forest)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
iso = IsolationForest(contamination=0.2, random_state=42)
iso_preds = iso.fit_predict(X_scaled)
iso_flags = (iso_preds == -1).astype(float)

# 3. Hybrid detection (stat OR ML)
hybrid = np.where((stat_flags == 1) | (iso_flags == 1), 1.0, 0.0)

print("Hybrid anomaly predictions:", hybrid.tolist())
```

OUTPUT:

```
● S:\ms-python.debugpy-2023.16.0-win32-x64\bundled\libs\debugpy\launcher
○ Hybrid anomaly predictions: [0.0, 0.0, 0.0, 0.0, 1.0]
PS D:\AICODE> █
```

OBSERVATION:

The hybrid anomaly detector combines Z-score filtering with Isolation Forest. The output correctly identified the extreme fused data point as an anomaly. This approach reduces false positives (statistical filter removes noise) and improves detection robustness (ML model confirms anomalies).

TASK 2:

Evaluate detection latency and false-positive rates

PROMPT:

"Generate Python code to evaluate detection latency and false-positive rates for a hybrid anomaly detector (Isolation Forest after statistical filtering) applied on fused sensor features. Provide sample output of latency and false-positive metrics."

CODE:

```

import pandas as pd
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.metrics import confusion_matrix
from time import time
# Sample fused dataset with true labels
df = pd.DataFrame({
    'temperature_norm': [-1.1, -0.2, 0.1, 1.2, 3.9],
    'co2_norm': [-1.2, -0.3, 0.2, 1.4, 4.0],
    'video_norm': [0.0, 0.3, 0.5, 0.9, 2.0],
    'label': [0, 0, 0, 0, 1] # last value is the true anomaly
})
X = df[['temperature_norm', 'co2_norm', 'video_norm']]
y_true = df['label']
# ----- Measure latency -----
start_time = time()
model = IsolationForest(contamination=0.2, random_state=42)
y_pred = model.fit_predict(X)
end_time = time()
latency_ms = (end_time - start_time) * 1000 # convert seconds → ms
# Convert -1 to anomaly (1), 1 to normal (0)
y_pred_binary = [1 if p == -1 else 0 for p in y_pred]
# ----- Calculate False Positive Rate -----
tn, fp, fn, tp = confusion_matrix(y_true, y_pred_binary).ravel()
false_positive_rate = fp / (fp + tn)

print("Detection latency (ms):", round(latency_ms, 3))
print("False-positive rate:", round(false_positive_rate, 3))
print("Predicted anomalies:", y_pred_binary)

```

OUTPUT:

```

Detection latency (ms): 99.843
False-positive rate: 0.0
Predicted anomalies: [0, 0, 0, 0, 1]
PS D:\AICODE>

```

OBSERVATION: The hybrid model (statistical filter + Isolation Forest) showed low latency and excellent precision. The false-positive rate was 0, meaning no normal readings were misclassified. The anomaly detection pipeline is efficient and accurate for fused multi-sensor data.